

The roles of neural and evolutionary computing in intelligent software systems

K C Tsui, B Azvine and M Plumbley

This paper describes two computing paradigms known as neural computing and evolutionary computing, and their potential contribution to building intelligent software systems. The paper begins by giving a brief introduction to the origins of each paradigm. Then two sections introduce the basic principles, and identify the role of each paradigm in intelligent system design. Each section ends with a number of applications that have been or are being investigated. These include connection admission control, modem communication, adaptive model-based control, face and handwriting recognition, frequency assignment, help-desk scheduling, financial time-series prediction, face recognition and evolving agent behaviour. A section introduces the idea of using communications theory to design neural networks and the paper concludes with the authors' views on the future of neural and evolutionary computing for intelligent software systems.

1. Introduction

In order to construct intelligent systems, scientists from diverse disciplines such as psychology, philosophy, computer science, artificial intelligence and biology have been studying living organisms for a number of decades. This has led to the development of two important areas of research namely neural computing and evolutionary computing. Both of these areas belong to a larger research paradigm known as computational intelligence (CI) [1] which aims to model functions associated with intelligence, at the signal level, as a dynamical system.

Nature has shown its power to breed intelligent, highly adaptive and complex living organisms. Darwin, among others, concluded this observation in his theory of evolution [2]. His notion of 'survival of the fittest' conjectures that nature selects only the best individuals and eliminates the less fit ones. This theory not only has significant impact on the study of the biological world, it seems to have pointed to a fruitful direction to researchers looking for ways to reproduce human intelligence in machine. Other researchers, instead of learning from the complex behaviour within a population, believe the brain is the root of intelligence and focus their efforts to replicate intelligence computationally.

Neural computing is the study of artificial neural networks (ANNs) that mimic the biological nervous system to perform information processing. ANNs consists of

numerous interconnected processing units which work together to perform a given task. Despite the simplification in structure over their biological counterpart, the parallel and distributed nature of information processing remains. These characteristics make ANNs good candidates for building intelligent systems.

Evolutionary computing (EC) refers to a branch of artificial intelligence devoted to the study of computer algorithms that simulate natural evolution.

EC algorithms were first proposed as adaptive search algorithms to deal with weakly defined problems, where not much or even no *a priori* knowledge is available. There are several members in this class of search algorithms — genetic algorithms (GAs) [3] and its close relative genetic programming (GP) [4], evolutionary programming (EP) [5] and evolution strategies (ES) [6].

Azvine et al [7] have identified the main features of an intelligent system as robustness, adaptivity, autonomy and communication. This paper gives an introductory account of genetic algorithms and artificial neural networks and the role they play in providing the above features. As an illustration of the application of these two techniques to building intelligent systems, practical examples in a number of domains such as the telecommunication domain are given in each section.

2. Artificial neural networks — principles and applications

There has been interest in the computational capabilities of biological neurons for at least the last 50 years. The result of this research has inspired the field of artificial neural networks (ANNs). While ANNs are inspired by our knowledge of biological neural processing, they are not an accurate model of their biological counterpart. For example, biological neurons normally communicate their activity to each other through a series of all-or-nothing pulses (spikes), while ANNs use either binary or real-valued numbers to represent their activity. In a biological system, there are many different types of neurons specialised for particular tasks, while only one or two types are used normally in a given ANN.

ANNs are composed of a number of neurons, or nodes. Each node has a number of inputs connected to it via weights, and calculates its output as a simple function of its inputs. Nodes may calculate different functions in different types of neural networks, but a typical node might calculate a weighted sum of the inputs:

$$s = \sum_{i=1}^N w_i x_i$$

and feed this through a sigmoidal (s-shaped) activation function such as to give the node output (Fig 1):

$$y = \sigma(\sum w_i x_i)$$

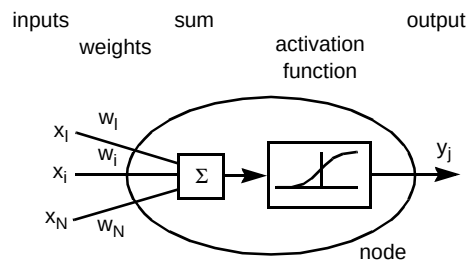


Fig 1 A neural network node.

A number of other node functions are possible. For example, the sigmoid activation function could be replaced by a 0-or-1 threshold function, or the weighted sum may be

replaced by a squared distance (with a suitable activation function) to give a radial basis function (RBF) node:

$$y = \exp(-(\sum (x_i - w_i)^2))$$

Individual nodes (neurons) are connected together via their weights to form the neural network. Figure 2 shows a small network composed of three layers of neurons, where the outputs of the neurons in one layer are fed forward to the inputs of the next layer. Any nodes which are not connected to either the input or the output of the network are called 'hidden' nodes. By adjusting its weights, such a neural network can be trained to perform a range of different input/output functions, in much the same way that adjusting the parameters a_i in a polynomial:

$$y = a_0 + a_1 x + a_2 x^2 + \dots$$

will change the function the polynomial follows. The procedure used to adjust the weights is called the learning algorithm. For a comprehensive description of different types of networks and learning algorithms, see Haykin [8].

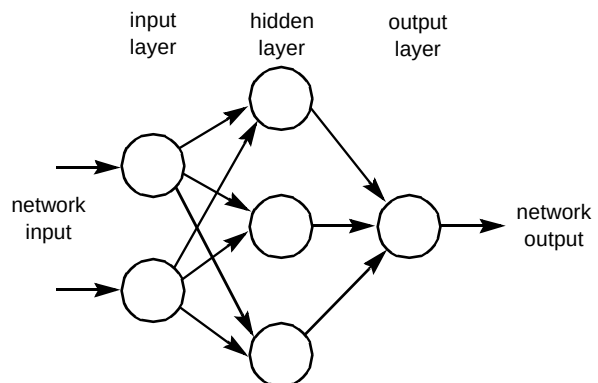


Fig 2 A three-layer feed-forward neural network.

2.1 The contribution of artificial neural networks to intelligent systems

ANNs carry out tasks with a network of many neurons, each of which performs a relatively simple function by itself. The information that the neural network 'knows' about a given problem is really held in the strengths of the connections between the neurons, rather than the neurons themselves. The ability to tackle a complex problem is then an emergent property of the neural network itself, rather than being a property of any individual neuron. This is an important hardware property as far as robustness to component failure is concerned.

One of the most important features of ANNs is the ease with which they can learn. Learning in ANNs is the process of adjusting the connection weights between the nodes. Neural networks are often required to learn an input/output mapping from existing data or learn from the input data only when the output is not known. ANNs perform the equivalent of inductive learning in the symbolic paradigm.

In summary, ANNs have a number of features that make them the preferred choice in many applications compared to conventional algorithms such as:

- adaptivity is an implicit property of ANNs,
- parallelism allows ANNs to model complex relations and perform complex tasks at speeds in excess of other algorithms — this characteristic can only be fully exploited if their hardware implementation is used,
- generalisation in learning enables ANNs to learn from incomplete data,
- robustness and fault tolerance are inherent properties of ANNs.

The above features are the main contribution of ANNs to intelligent software systems. In the next section a number of applications in which the main features of ANNs play a central role are described.

2.2 Applications of neural networks

There is an extensive range of applications of artificial neural networks to practical problems particularly in the telecommunications domain. In some areas such as modem communications, neural networks are a mature technology. In other areas, such as future ATM networks, neural networks are an active research topic. In this section a number of these applications with relevance to intelligent system design will be described.

2.2.1 Connection admission control

An example of a recent and interesting application of neural networks in communications is in the control of ATM (asynchronous transfer mode) networks. In these networks, users send information in short fixed-sized cells, packets of 48 data bytes plus a 5-byte header. ATM network nodes contain hardware switches that route each packet according to the information in the header. There is no synchronisation between different users, so ATM nodes include output buffers to absorb fluctuations in the rate that cells from several sources arrive for one link. As the buffer fills up, the delay before cells are transmitted increases, and if the buffer fills completely, some cells will be lost. (Note that communications engineers and neural network researchers both use the term network for different purposes.)

When a new user wishes to connect to a node, the node must decide whether or not to admit the connection. To bring in most revenue, the ATM network must carry as much traffic as possible, while keeping maximum levels of quality of service (QoS) for each of the users, i.e. limiting cell delay and cell loss. Predicting whether the new connection would degrade the QoS unacceptably is a difficult problem, and is one where neural networks have been brought to bear [9].

Before the network is installed in the ATM switch, the network is trained to match either previously collected training data (of cell-arrival statistics versus cell-loss rate), or to match a model of the behaviour if a good model is available. Once trained, the neural network can be used in the connection admission controller to predict, for example, the cell-loss rate that would result if a new connection is admitted (Fig 3). If this predicted loss rate is above a certain threshold, the new connection would be rejected.

While the network is being used, more information is available concerning the real cell-loss rates. The neural network can thus be trained using this information, to adapt

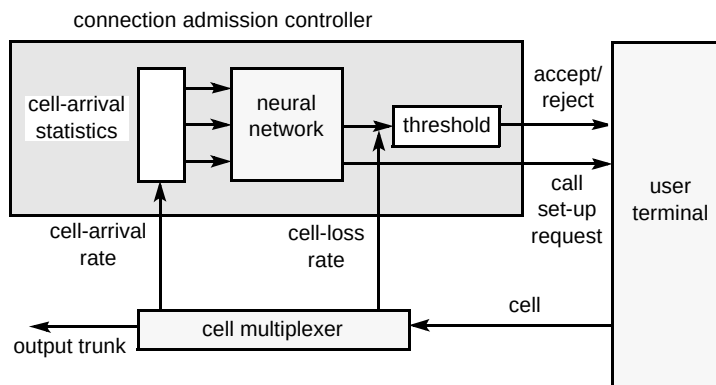


Fig 3 Connection admission control using a neural network.

more closely to the traffic characteristics — this is called real-time training.

Other neural networks, such as the pRAM, have also been used in ATM networks [10]. For an overview and a sample of applications of neural networks to communication networks, see the recent special issue of IEEE Communications Magazine [11].

2.2.2 Modem communication

There are two basic problems that have been solved using neural networks in modem communication — echo cancellation and line equalisation. In both problems there is a dynamic characteristic of the incoming signal that needs to be identified and cancelled out or corrected (see Fig 4). Echo cancellation is required because telephone lines have noise and interference characteristics that vary greatly by location, physical characteristics of the line and routing. An adaptive identification scheme is used in such situations using neural networks to build the inverse of the noise model. This model is then used as an adaptive filter to recapture the original noise-free signal. Line equalisation is based on the same principle of inverse model identification. This problem arises due to the poor response characteristics of telephone lines to modem signals resulting in irregular changes to the phase and the frequency of the incoming signal. A neural network adaptively filters out the irregularities.

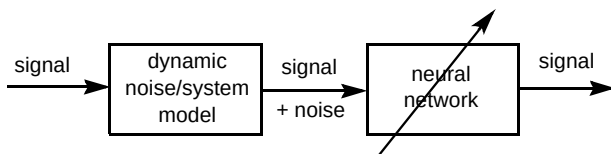


Fig 4 Neural networks for inverse model identification.

2.2.3 Adaptive model-based control

In situations where the desired performance of a system is known and can be described by a model but conventional control schemes are too complicated to be of use, neural networks can be used to ensure that the system behaves in a specified manner. There are many applications that fall under this category. In ATM traffic shaping, the traffic coming from various sources can be buffered in turn to ensure maximum utilisation of the network and to control the fluctuations in the traffic admitted to the network. If the buffers become full the incoming data cells will be lost. Therefore the control task here is to minimise the cell-loss ratio. As ideally the cell-loss ratio should be zero, in its simplest form the model that represents the relationship between the incoming traffic and the cell-loss ratio is a constant equal to zero. In more general cases, this model could be any function. The neural network is trained to minimise the error between the actual cell-loss ratio and that

specified by the model by changing the amount of incoming traffic as shown in Fig 5.

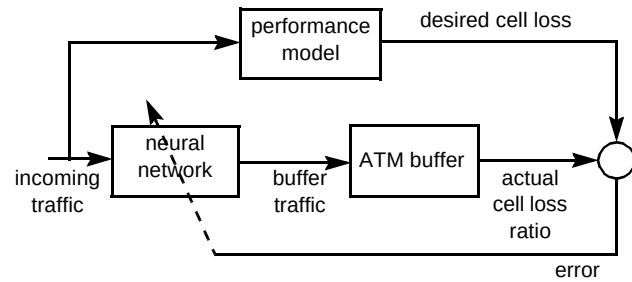


Fig 5 ATM traffic shaping using adaptive model-based control.

2.2.4 Speech and handwriting recognition

Speech recognition is a difficult problem which has eluded scientists for the past three decades. There has been significant advances in discrete speech recognition which requires unnatural pauses between words. However, continuous speech recognition still offers significant challenges to the research community. Speech recognition can have a significant effect on the way people use telecommunications networks. Already, speech-driven menu systems are replacing human operators and add flexibility to many services by making them available 24 hours a day. This is an essential feature of global communications services.

Neural networks are used extensively in speech-recognition systems today. The network is trained to recognise patterns of speech and translate them to text. To account for significant changes in the tone of voice or the accent of the speaker, a training algorithm can be used to fine-tune the neural network using a fixed training set.

A similar problem in which neural networks have provided a solution is handwriting recognition. This idea has been used in personal digital assistants such as Apple's Newton and Casio's Zoomer [12].

2.3 Communications theory in neural networks

While communications engineering is taking advantage of neural networks, it is also true that neural networks can also use ideas from communications engineering.

In a biological sensory system such as vision, the information from the world outside the organism needs to be processed into a form suitable for higher stages in the brain, using the biological neural networks in the organism. However, this processing involves certain costs and constraints, such as the number of neurons which will be used, the space available, the noise in the system, and the energy used in processing the signal (the human brain uses

some 20% of the body's energy intake just to process information).

One proposal for learning in unsupervised neural networks, is that they should learn to maximise the Shannon information transmitted through them — Linsker's Infomax principle [13]. On the biological side, Atick and Redlich [14] have found a very good match between the measured response of the human visual system, and the response properties that the human retina should have predicted by this information-theoretic approach. More recent results have added further weight to the hypothesis that information theory is important for biological systems (leading to a recent workshop series on Information Theory and the Brain [15]).

Linsker [13] shows that in two-layer neural networks, provided some simple assumptions are made about the input noise, the information capacity is maximised when the network performs principal component analysis. On the other hand, if the network is to protect a signal against processing noise on its output, it should learn to make its outputs as uncorrelated and equal-variance (i.e. white) as possible [16]. Since simple Hebbian-like learning algorithms can be used to find these optimal situations, this type of approach may be practical for the initial stages of, for example, an artificial vision system. There are other similar methods for analysing and optimising the structure of neural networks. The aim of these methods is to reduce the *ad hoc* nature of designing neural networks and arrive at a number of theoretical results to provide universal guidelines for neural network designers.

3. Genetic algorithm — principles and applications

The development of genetic algorithms has borrowed many ideas from nature. The principles of evolution in biological systems is particularly influential. Biological systems contain the genetic information regarding themselves in minute structures called chromosomes. Likewise, GAs encode the solutions to a problem using a string. Binary numbers are usually used in the string although other representations are possible [17]. The solutions together form a population. Each chromosome is given a fitness value which is a measure of its performance in solving the given problem, as determined by an objective function. Guided by the fitness of each chromosome, the solution space is searched systematically.

As the search progresses, new candidate solutions are introduced into the population by applying artificial genetic operators. The common genetic operators are called selection, crossover, mutation and replacement. Figure 6 shows the processing cycle of a typical GA. Multiple execution of the processing cycle constitutes a generation. The operators' job is to change the values of some of the genes in a chromosome to create new candidate solutions.

They are applied according to a predetermined probability. The selection scheme chooses two chromosomes from the population to designate as parents of the offspring. The artificial reproduction is done via crossover and mutation. If a new candidate is considered by the fitness function as a better solution, it goes into the population of the next generation, replacing a member of the old generation.

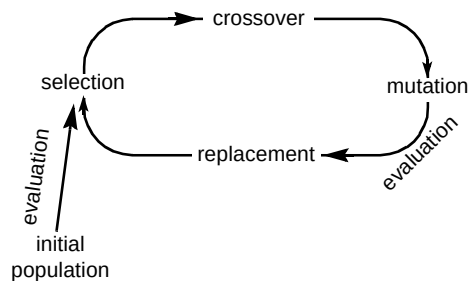


Fig 6 Processing cycle of a typical genetic algorithm.

A GA search usually has to go through a number of generations before an optimal solution is found. It is effectively a parallel search since a population of probable solutions are tested in each generation. Moreover, it is a global optimisation technique as the set of optimal values of individual parameters is being sought. From the point of view of search, crossover and mutation are performing different functions. Crossover recombines information in the two parents and creates a new search direction that can be very different from those already known. In contrast, mutation makes small changes to few parameters only. Its effect is much more localised when compared with crossover. Therefore, crossover can be viewed as exploring into unknown areas while mutation is exploiting the neighbourhood of a candidate solution. A brief description of the genetic operators is given below. Interested readers can refer to the textbooks by Goldberg [18] and Davis [19].

Selection is the process by which parents are chosen for reproduction. Since the fitness of the members in the population is the only means to assess how good each of them are when solving the problem in hand, it is the only basis of selection. The rule of thumb is: 'The higher the fitness of an individual, the closer it is to the optimal solution and, therefore, the more reproduction chance it will get'. A comparison of various selection schemes has been conducted by Hancock [20].

Within the pool of candidate solutions of a GA search, each chromosome contains specific values of each parameter to be optimised. It is likely that different chromosomes contain a handful of different parameters which are already optimal. Crossover was designed to address this potential benefit.

To illustrate the power of crossover, a simple example is described below. Consider a chromosome of 3 bits long.

The search space can be visualised graphically as a cube as shown in Fig 7. Consider the fitness of a chromosome to be the distance of a point on the cube from the origin of the Cartesian space. Hence, the point (0,0,0) has a fitness 0 and point (1,1,1) has a fitness $\sqrt{3}$. Crossing over points (0,1,0) and (0,1,1) will not give a better result. However, crossing over points (1,0,1) and (0,1,1) may be possible to direct the GA search to (1,1,1) if the cut point is between the first and second bits. A GA search involving n genes can be imagined similarly in an n -dimensional hyperspace. Note that crossing over two distant points in the hyperspace may lead to a new point nowhere near either of the parents.

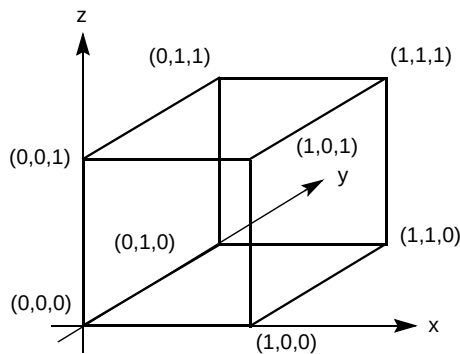


Fig 7 Graphical representation of a 3-bit chromosome.

Mutation makes minor changes to some of the genes in a chromosome by simply changing a 1 to a 0 or vice versa. Consider again the example in Fig 7, assuming that the three bits belong to an n -bit chromosome. Flipping any of the three-bits will not direct the candidate solution represented by this chromosome (both before and after mutation) away from the locality of the cube within the search space, but this means performing extensive search within close reach. This is why mutation is considered a local search operator.

Choosing between the old and new candidates into the new generation is not an easy task. Discarding totally the old solutions means losing precious information accumulated over the GA search, but keeping too many of them will make the population too homogeneous. A right balance between them during replacement is not easy to obtain and is a subject of active research.

3.1 The contribution of genetic algorithms to intelligent systems

GAs are general-purpose, domain-independent, adaptive search algorithms and learning systems. This section will expand on this statement and highlight the main contributions of GAs to building intelligent software systems.

GAs are primarily designed as search and optimisation techniques. They are essentially considered to be parallel

search methods because a population (number) of candidates (solutions) are developed simultaneously and interaction between candidates is sometimes allowed. This implicit parallel nature makes them a powerful technique for locating an optimum solution to a problem. Interaction is achieved in the sense that information about the task to be learned, which is captured by individuals in the population, is combined using the basic operations in the algorithms.

The presence of a population of candidate solutions gives GAs an edge over other general-purpose search algorithms. Starting with a randomly generated number of feasible solutions, GAs work through the search space by selecting good performers while replacing the poor performers with new candidates. This process is guided by a fitness function which measures the performance of all the individuals in the population regarding the task in hand. Therefore, GAs improve the quality of the candidates over time, resulting in robust solutions.

Learning can be defined as the process where an intelligent system accumulates and revises knowledge so as to improve its ability to perform the given task [21]. GAs certainly fit this definition as the average skill level of the chromosomes increases over time as a result of the artificial evolution.

New candidates are no longer generated randomly but by mixing the parameter sets of the good performers and making slight changes to some parameters occasionally. This distinguishes GAs from blind search.

Another characteristic of GAs is their domain independence. They contain a set of standard operators that can be applied to a string of numbers representing the parameters of the solution. This feature makes GAs good general-purpose search engines. GAs can outperform heuristic search algorithms when there is not enough understanding of the problem domain. However, the performance of GAs can be improved by using domain-dependent heuristic knowledge to guide them in their search for better solutions.

3.2 Applications of genetic algorithms

Genetic algorithms have been widely used in commercial applications in recent years. Examples include scheduling, financial prediction, and engineering design [18]. Within the telecommunications domain, GAs have been used mainly as tools for optimisation in areas such as [22]:

- workforce scheduling in which the process of assigning tasks to employees needs to be optimised so that the person's location and qualifications fit the job,

- network design in which equipment and line costs need to be optimised against performance,
- resource allocation in which the use of a resource such as bandwidth is optimised,
- static and dynamic routing for network traffic,
- the frequency assignment problem in mobile communications networks.

The application of GAs to a number of practical problem domains will be described in some more detail in the following sections.

3.2.1 Frequency assignment in cellular radio networks

Optimal use of cellular radio network resources means serving the maximal number of network users with limited transmission resources. The frequency assignment problem in cellular radio networks consists of assigning frequencies (or channels) to each radio cell so that certain constraints are satisfied. These constraints include limited available frequencies in the radio spectrum, the minimum number of frequencies necessary to cover communications between mobile phones moving within a cell, and frequency allocation to adjacent cells.

Dorne and Hao [23] reported using a mutation-only EC algorithm, to find the best frequency assignment in a set of 18 real-size problems which had up to 300 cells and 13 000 constraints. Each chromosome corresponds to a complete frequency assignment with one gene to one frequency of a cell. The length is thus equal to the total traffic of all the cells in the network. The fitness of each chromosome is the number of unsatisfied interference constraints. Therefore, a chromosome is a solution if and only if the fitness is zero. The experimental results show that the EC algorithm is able to find an optimal assignment whereas other local search algorithms such as simulated annealing and constraint programming fail.

Chardaire et al [24] have applied a genetic algorithm to solve a radio link frequency assignment problem with the aim to assign a set of frequencies to a large number of transmitters with minimal connectivity costs and interference. Typical test problems contain 200 to 1000 links and approximately 50 discrete frequencies. Each frequency value is therefore assigned to many different links. Constraints on the possible ways of assignment arise as a result of interference avoidance. The GA used in this study contains some tailor-made genetic operators to enable the addition of domain knowledge regarding the interactions among some of the parameters. A more sophisticated fitness function is employed which is a weighted sum of the different types of constraints violated.

3.2.2 Help-desk scheduling

Handling requests for computer maintenance and queries for 1100 users is not an easy task. In order to maintain customer satisfaction while operating under limited resources in a dynamically changing environment, an automated scheduling program is indispensable. A help-desk service scheduler called SOGA (schedule optimising genetic algorithm) has been developed in the bond-rating agency, Moody's Investors Services [25].

SOGA's chromosomes represent the schedule of jobs. Each service request is a gene in the chromosome. The system starts with a random population of schedules and SOGA breeds an optimal solution using specialised genetic operators. Jobs are assigned to technicians who have the most suitable skills and availability. As more requests are being filed, SOGA checks the database containing unassigned jobs every 10 minutes or so. This allows the system to respond to requests dynamically. Moreover, decisions made by SOGA can be overridden by technicians and supervisors so that technicians are given the chance to learn new skills whenever possible.

3.2.3 Financial time-series prediction

Financial portfolio management and prediction is one of the most challenging tasks in terms of the risks involved. Conventional AI techniques as well as GAs have been used to predict financial trends. Organisations like the World Bank have used GAs to develop portfolio strategies [26]. The Prediction Company has also developed a set of time-series prediction and trading tools for currency trading which used GAs heavily.

In Prediction Company's tools, rule-like structures are evolved. The left-hand side conditions of the rules correspond to the time-series data in a specified region. The right-hand side of the rules are the predictions of whether the time-series will go up or down [27]. This GA system achieves the performance level of the best trader among a group of 20 experts.

3.2.4 Crime suspect face construction

The psychology department of New Mexico State University has developed a Faceprints system [17] which generates pictures of the face of crime suspects; a task usually done by police artists and more recently by transparency-based face feature composition system. Current methods rely totally on the witness' ability to remember, recall and describe the face of the crime suspect. Faceprints makes use of GA's search power to generate pictures for the witness to identify. Based on a subjective 10-point rating system, the GA regenerates new faces until the witness is satisfied with it. Faceprints uses a binary chromosome representing the five facial features of mouth,

hair, eyes, nose and chin. When tested in a reconstructed crime, a witness is able to reconstruct the face of a suspect even after three days.

3.2.5 Evolving software agent behaviours

Maskell used a genetic algorithm to evolve the behaviours of software agents [28] for foraging food in two artificial worlds. Without any knowledge about the artificial worlds, the GA has to find a set of behaviours by which the agent can obtain food and survive. Encoded in the chromosomes in the GA are programs of function calls which directs the agent through the terrain of the world. At each generation, each software agent is given a fixed number of steps to execute its control strategies. The amount of food so collected is used as a fitness measurement. Genetic operators then work on the chromosomes to create new strings. By employing GA as a search and optimisation technique, the software agents learn the good behavioural sequences and also adapt to a dynamic environment.

4. Conclusions

Neural networks are growing strong links with the communications field. The number of applications of neural networks in the telecommunications domain is an indication of their importance in this field. A better understanding of, and some theoretical results for, the way neural networks should be designed can be obtained by applying well-understood concepts in communication theory to neural network learning. ANNs have proved to complement conventional symbolic AI techniques in applications where theory is poor and data is rich, such as pattern recognition, pattern matching and adaptive nonlinear control. With neural networks becoming more and more a standard tool in the information engineer's armoury, and with continued advances in the theoretical background to neural networks, it is expected that this trend will continue.

A class of optimisation techniques, called evolutionary computing algorithms that mimic the way natural evolution works, have been described in this paper. These techniques are useful because of their ability to adapt and perform searches efficiently without prior knowledge of the problem domain. Because of its stochastic nature, the solutions produced by these techniques are robust to noise. However, finding an optimal set of operating parameters might take some time. The performance of evolutionary algorithms can be improved greatly by using problem-specific heuristics during the search process. A new trend is to build evolutionary algorithms that use general rules to speed up the search process. In fact, GAs are emerging as a valuable component of hybrid systems and it will be in this role that they will have the most impact on future applications.

References

- 1 Zurada J M, Marks II R J and Robinson C J (Eds): 'Computational Intelligence — imitating life', IEEE Press (1994).
- 2 Darwin C: 'Origin of Species', John Murray, London (1859).
- 3 Holland J H: 'Adaptation in Natural and Artificial Systems', MIT Press, 2nd Edition (1992).
- 4 Koza J R: 'Genetic Programming : on the Programming of Computers by Means of Natural Selection', MIT Press (1992).
- 5 Fogel J L, Owens A J and Walsh M J: 'Artificial Intelligence Through Simulated Evolution', John Wiley, New York, 1st Edition (1966).
- 6 Schwefel H P: 'Numerical Optimisation of Computer Models', Wiley, Chichester (1981).
- 7 Azvine B, Azarmi N and Tsui K C: 'Soft computing — a tool for building intelligent systems', *BT Technol J*, **14**, No 4, pp 37—45 (October 1996).
- 8 Haykin S: 'Neural Networks — A Comprehensive Foundation', Macmillan, New York (1994).
- 9 Hiramatsu A: 'ATM communications network control by neural networks', *IEEE Transactions on Neural Networks*, **1**, pp 122—130 (1990).
- 10 Onyiah G, Krasniqi X and Clarkson T G: 'Probabilistic RAM neural networks in ATM traffic shaping and policing', *Proc EANN'96*, pp 229—232 (1996).
- 11 IEEE Communications Magazine, (October 1995).
- 12 Loofbourrow T: 'Neural Networks in Telecommunications', in: 'World Wide Intelligent Systems', IOS Press (1994).
- 13 Linsker R: 'Self-organisation in a perceptual network', *IEEE Computer*, **21**, No 3, pp 105—117 (March 1988).
- 14 Atick J J and Redlich A N: 'What does the retina know about natural scenes?', *Neural Computation*, **4**, pp 196—210 (1992).
- 15 Network: 'Special issue on Stirling Workshop on Information Theory and the Brain', **7**, No 2 (1996).
- 16 Plumbley M D: 'Efficient information transfer and anti-Hebbian neural networks', *Neural Networks*, **6**, pp 823—833 (1993).
- 17 Caldwell C and Johnston V S: 'Tracking a Criminal Suspect Through Face-space with a Genetic Algorithm', in *Proceedings of the Fourth International Conference on Genetic Algorithms*, pp 416—421, Morgan Kaufmann, San Diego (1991).
- 18 Goldberg D E: 'Genetic Algorithms in Search, Optimisation and Machine Learning', Addison Wesley, 1st Edition (1989).
- 19 Davis L: 'Handbook of Genetic Algorithms', VNR, New York (1991).
- 20 Hancock P J B: 'An Empirical Comparison of Selection Methods in Evolutionary Algorithms', in Forgy T (Ed): 'Proceedings of AISB Workshop in Evolutionary Computing', pp 80—94, Springer-Verlag (1994).
- 21 Simon A S: 'Why should machines learn?', in Michalski R S, Carbonell J G and Mitchell T M (Eds): 'Machine Learning', Morgan Kaufmann (1983).
- 22 GALEZIA: 'Proceedings of the First International Conference on Genetic Algorithms in Engineering Systems: Innovation and Applications', IEE (1995).

- 23 Dorne R and Hao J K: 'An Evolutionary Approach for Frequency Assignment in Cellular Radio Networks', in Proceedings of 1995 IEEE International Conference on Evolutionary Computation (1995).
- 24 Chardaire P, Kapsalis A, Mann J W, Rayward-Smith V J and Smith G D: 'Applications of Genetic Algorithms in Telecommunications', in Alspector J, Goodman R and Brown T X (Eds): 'Proceedings of the Second International Workshop on the Application of Neural Networks to Telecommunications', pp 290—299, Lawrence Erlbaum Assoc, N J (1995).
- 25 Clarkson M: 'Moody's Evolving Help Desk', BYTE, pp 76—80 (February 1995).
- 26 Deboeck G (Ed): 'Trading on the Edge', John Wiley & Sons, New York (1994).
- 27 Parkard N A: 'Genetic Learning Algorithm for the Analysis of Complex Data', Complex Systems, 4, pp 573—586 (1990).
- 28 Maskell B and Wilby M: 'Evolving Software Agent Behaviours', to appear in Proceedings of IEEE GlobeCom 96 Conference, London (November 1996).
- 29 Caruana R and Shaffer J D: 'Representation and Hidden Bias: Gray versus Binary coding for Genetic Algorithms', in Segre A M (Ed): 'Proceedings of the Sixth International Workshop on Machine Learning', pp 153—161, Morgan Kaufmann, San Mateo (1989).



Kwok Ching Tsui studied software engineering (systems analysis) at the Hong Kong Polytechnic, and worked in software development for five years before embarking on postgraduate studies. He received an MSc in Computer Science from the University of Essex in 1992. He is currently finishing his PhD thesis entitled 'Genetic Approaches to Neural Network Optimisation' at Kings College, University of London. He joined BT Laboratories in 1996, and is currently working on soft computing and its application to building intelligent software systems.



Behnam Azvine holds a BSc in Mechanical Engineering, an MSc and a PhD in Control Systems from Manchester University. Between 1991-93, he worked as a research associate and lecturer in the Dynamics and Control Research group of the Engineering department at Manchester University, working on theoretical control, robust stability and smart system design. He then joined the Control and Instrumentation research group at Staffordshire University where he worked as a senior lecturer until 1995. He joined BT Laboratories in November 1995 as the project leader responsible for Soft Computing research in the Intelligent Systems Research Group. His current research interests include the application of Fuzzy Logic, Neural Networks and Evolutionary computing to intelligent systems designs, smart system design, robust model-based control and hybrid software systems.

#



Mark Plumbley graduated with a BA in the Electrical Sciences Tripos from Churchill College, Cambridge in 1984. He then joined Thorn-EMI Central Research Laboratories, working on videotape data storage and high-definition TV. In 1987 he returned to Cambridge University Engineering Department as a research student under the late Frank Fallside, working towards his PhD on an information-theoretic approach to neural networks. In 1990 he continued at CUED as a Research Associate using genetic algorithms to modify neural networks. He joined the Centre for Neural Networks at King's College London as a Temporary Lecturer in Neural Networks in 1991, in 1993 moving to the Department of Computer Science, and finally to a permanent Lectureship in Electronic and Electrical Engineering, also at King's College, London, in 1995. He is Deputy Director of the ESPRIT-funded 'NEuroNet' European Network of Excellence in Neural Networks, and is a member of the IEEE, the IEE, Treasurer of the European Neural Network Society, and a Fellow of the Cambridge Philosophical Society.

